

Course: JavaScript

Week:03

Department: Computer Science

We live under the shared quote

“Batch 3 - 3 Departments”

Supported by:

CADT
IDT វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology

Organized by:

NEKT
GEN-3
Batch 3, 3 Departments



What We Will Cover in 5 Weeks

Git & GitHub

W1

Introduction to JavaScript

W2

Array Functions & Objects

W3

Document Object Model

W4

Browser Events

W5



TABLE OF CONTENTS

Function 01

Array 02

Objects 03

Function

What is a Function?

$$f(x)$$

Functions are the main “building blocks” of the program.

They allow the code to be called many times without repetition

```
1  function showMessage(from, text = "no text given") {  
2      alert(from + ": " + text);  
3  }  
4  
5  showMessage("Ann"); // Ann: no text given
```

keyword

function name

parameters

call/invoke a function

Declaration vs Expression

A Function Declaration can be called earlier than it is defined.

```
1 console.log(sum(1, 2)); // 3
2
3 function sum(x, y) {
4     |     return x + y;
5 }
```

A Function Expression is created when the execution reaches it and is usable only from that moment.

```
1 console.log(sum(1, 2)); // error
2
3 const sum = function (x, y) {
4     |     return x + y;
5 }; // no more magic
```

In most cases when we need to declare a function, a **Function Declaration is preferable**, because it is visible prior to the declaration itself. That gives us more flexibility in code organization, and is usually more readable.

Arrow Function

There's another very simple and concise syntax for creating functions, that's often better than Function Expressions. It's called **arrow functions**

```
1  const sum = (a, b) => a + b; // more than one args
2
3  const double = n => n * 2; // you can do this with one arg
4
5  const sayHi = () => console.log("Hello!"); // no arg
6
7  const sub = (a, b) => { // the curly brace opens a multiline function
8      const result = a - b;
9      return result; // if we use curly braces, then we need an explicit "return"
10 };
```

Arrow functions are **handy for simple actions**, especially for one-liners. They come in two flavors: With curly brace and without curly brace

Callback Function

A **callback function** is a function passed as an argument into another function, is intended to be **executed later**.

```
1 function greet(name, callback) {  
2     console.log("Hello, " + name);  
3     callback(); // The callback is executed here  
4 }  
5  
6 function sayGoodbye() {  
7     console.log("Goodbye!");  
8 }  
9  
10 // Passing sayGoodbye as a callback  
11 greet("Alice", sayGoodbye);
```

No Invocation ()
just pass the name
of the function

Array in Javascript

Array is used to store an ordered collection of multiple values under a single variable name

Declaration and accessing array

```
1  // declare array
2  const arr = [];
3  const fruits = ["Apple", "Orange", "Plum"];
4  console.log(fruits[0]); // Apple
5  console.log(fruits[1]); // Orange
6  console.log(fruits[2]); // Plum
7  console.log(fruits.length); // 3
8  console.log(fruits.at(1)); // orange
9  console.log(fruits.at(-1)); // plum
10
11 // An array can store elements of any type in JS
12 const arrMix = ["Apple", 11, true];
```



Array in Javascript

Modifying an array

```
1  const fruits = ["Apple", "Orange", "Plum"];
2  fruits[2] = "Pear"; // now ["Apple", "Orange", "Pear"]
3  fruits[3] = "Lemon"; // now ["Apple", "Orange", "Pear", "Lemon"]
```

Common Array Operations

push appends an element to the
end

```
5  fruits.push("Banana");
6  // now ["Apple", "Orange", "Pear", "Lemon", "Banana"]
```

pop removes the last element from an array and returns
that element

```
8  console.log(fruits.pop()); // "Banana"
9  // now ["Apple", "Orange", "Pear", "Lemon"]
```

shift extracts the first element of the array and
returns it

```
11 console.log(fruits.shift()); // "Apple"
12 // now ["Orange", "Pear", "Lemon"]
```

unshift adds the element to the beginning of
the array

```
14 fruits.unshift("Strawberry");
15 // now ["Strawberry", "Orange", "Pear", "Lemon"]
```

Array in Javascript

Common Array methods

arr.forEach() method allows to run a function for every element of the array

```
1 ["Smey", "Sovath", "Bora"].forEach((item, index, array) => {  
2   |   console.log(`${item} is at index ${index} in ${array}`);  
3   | });
```

arr.map() method calls the callback function for each element of the array and returns the array of results

```
5 const lengths = ["Smey", "Sovath", "Bora"].map((item) => item.length);  
6 console.log(lengths); // [4, 6, 4] it return a new array
```

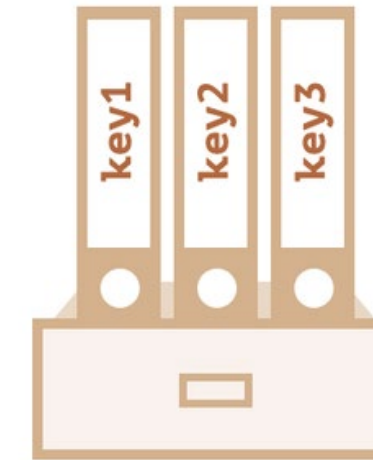
arr.filter() method creates a new array containing all elements that passed the callback function conditions

```
1 const numbers = [1, 5, 8, 12, 4];  
2 // Filter for numbers greater than 7  
3 const result = numbers.filter((num) => num > 7);  
4 console.log(result); // [8, 12]
```

There are more array methods but these are the essential and commonly used ones

Object in Javascript

Object is a collection of related data and/or methods, represented as a list of key-value pairs



Creating an object

```
1 let user = new Object(); // "object constructor" syntax
2 let user = {}; // "object literal" syntax
```

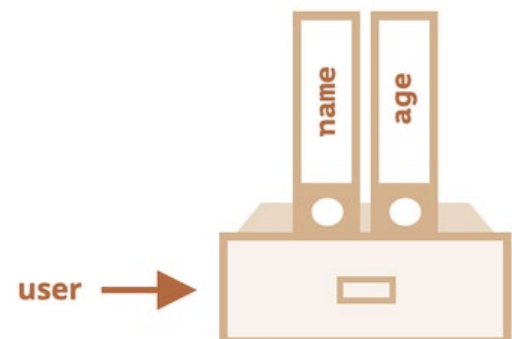


Getting values

```
1 // get property values of the object:
2 alert( user.name ); // John
3 alert( user.age ); // 30
```

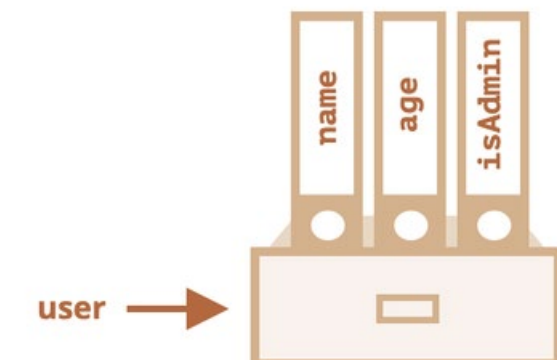
Assigning properties

```
1 let user = { // an object
2   name: "John", // by key "name" store value "John"
3   age: 30 // by key "age" store value 30
4 };
```



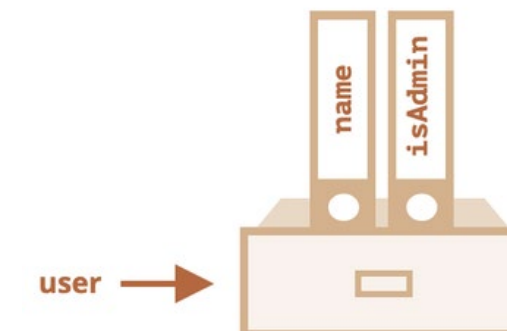
Adding properties

```
1 user.isAdmin = true;
```



Deleting properties

```
1 delete user.age;
```



Object in Javascript

Square Bracket Notation

```
1 // this would give a syntax error
2 user.likes birds = true
```

This dot notation doesn't work here. JavaScript doesn't understand that. It thinks that we address `user.likes`, and then gives a syntax error when comes across unexpected `birds`.

For **multi-word** keys we use square **bracket notation** `[]` instead

```
1 let user = {};
2
3 // set
4 user["likes birds"] = true;
5
6 // get
7 alert(user["likes birds"]); // true
8
9 // delete
10 delete user["likes birds"];
```

Checking if an object contains a key

```
1 let user = { name: "John", age: 30 };
2
3 alert( "age" in user ); // true, user.age exists
4 alert( "blabla" in user ); // false, user.blabla doesn't exist
```

Looping an object (for .. in)

```
1 let user = {
2   name: "John",
3   age: 30,
4   isAdmin: true
5 };
6
7 for (let key in user) {
8   // keys
9   alert( key ); // name, age, isAdmin
10  // values for the keys
11  alert( user[key] ); // John, 30, true
12 }
```

Thank You!

